REAL-TIME SCAM CALL DETECTION

Multimodal Detection Using Conversation Stage Modeling and Ensemble Learning

Pritam Paul · Arya Gupta · Srijan Sarkar · Shinjan Saha · Subhalaxmi Chakraborty

Institute of Engineering & Management · University of Engineering and Management, Kolkata

github.com/CyberKnight-cmd/spam-call-detection

IEM · UEM

Outline

- Introduction and motivation
- Signal processing: cleaning audio, MFCC features
- CNN fundamentals: the maths
- The four models, one by one
- Ensemble: weighted fusion and EMA smoothing
- Results and discussion
- Conclusion and references

Introduction

Why scam calls need real-time detection

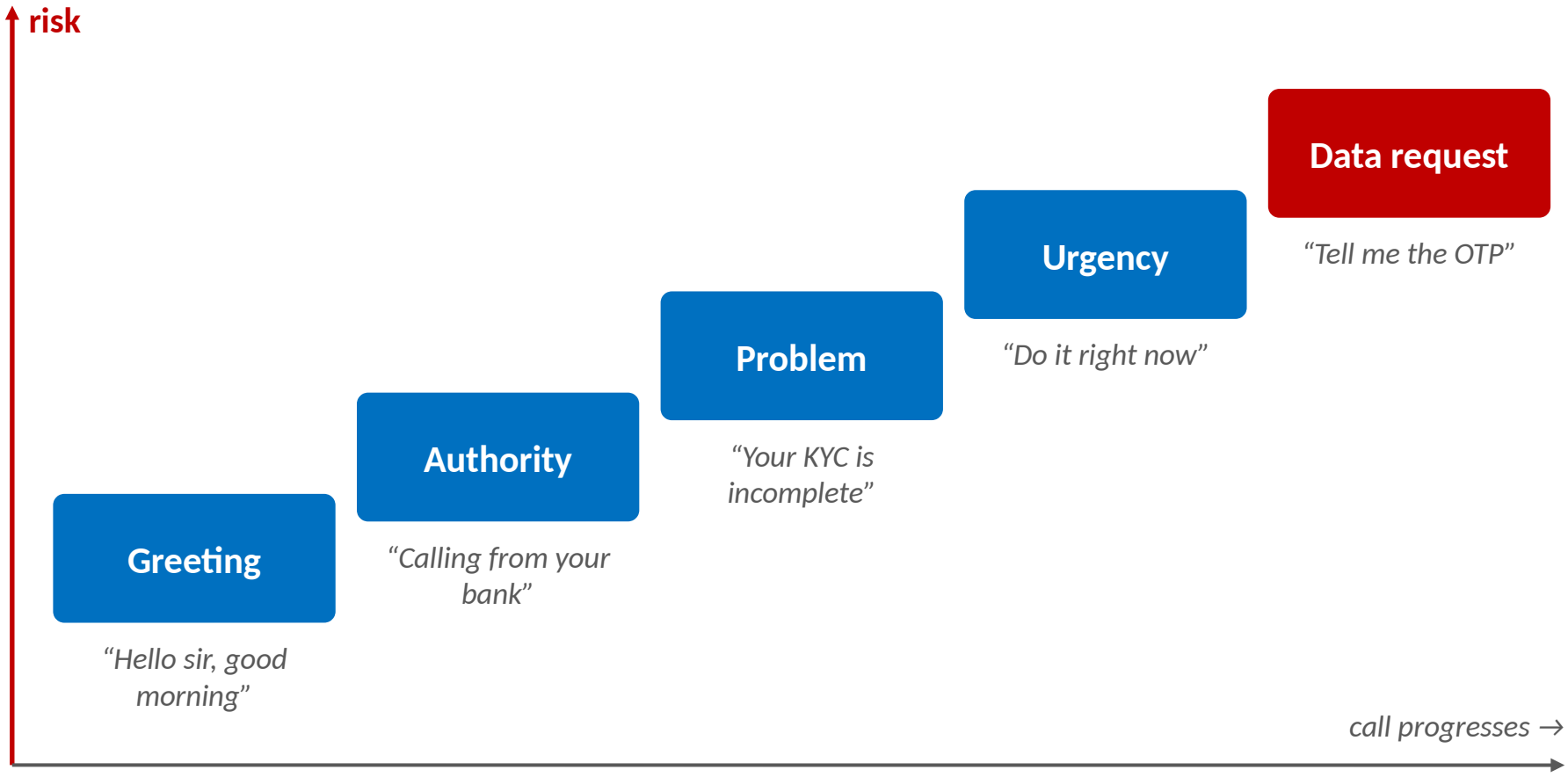
Why Scam Calls Are a Problem

- **Social engineering:** a person is persuaded; nothing is hacked
- **Goal:** OTP, bank details or money
- **Hard to block:** fluent speech, new numbers every day
- **Damage happens during the call**

Key question

Can we warn the user while the call is still going on?

How a Scam Call Unfolds



Scam calls follow a fixed script [6]. Know the stage → know the danger.

Related Work and the Gap

Approach	Main idea	Limitation
Blacklists & rules [1], [2]	Block known numbers; rules on call duration	Numbers change; false alarms
Call-record ML [3], [4]	Clustering / DTW on call statistics	Never listens to the call
Content analysis [5], [6]	NLP / deep learning on transcripts	After the call; one modality

Gap: no real-time system combines voice + text + conversation order.

Our Contributions

1 **Conversation-stage modelling**

Track the scam script live

2 **Four-model ensemble**

Phoneme, prosody, repetition, stage

3 **Weighted fusion + EMA**

A stable risk score every 4 s

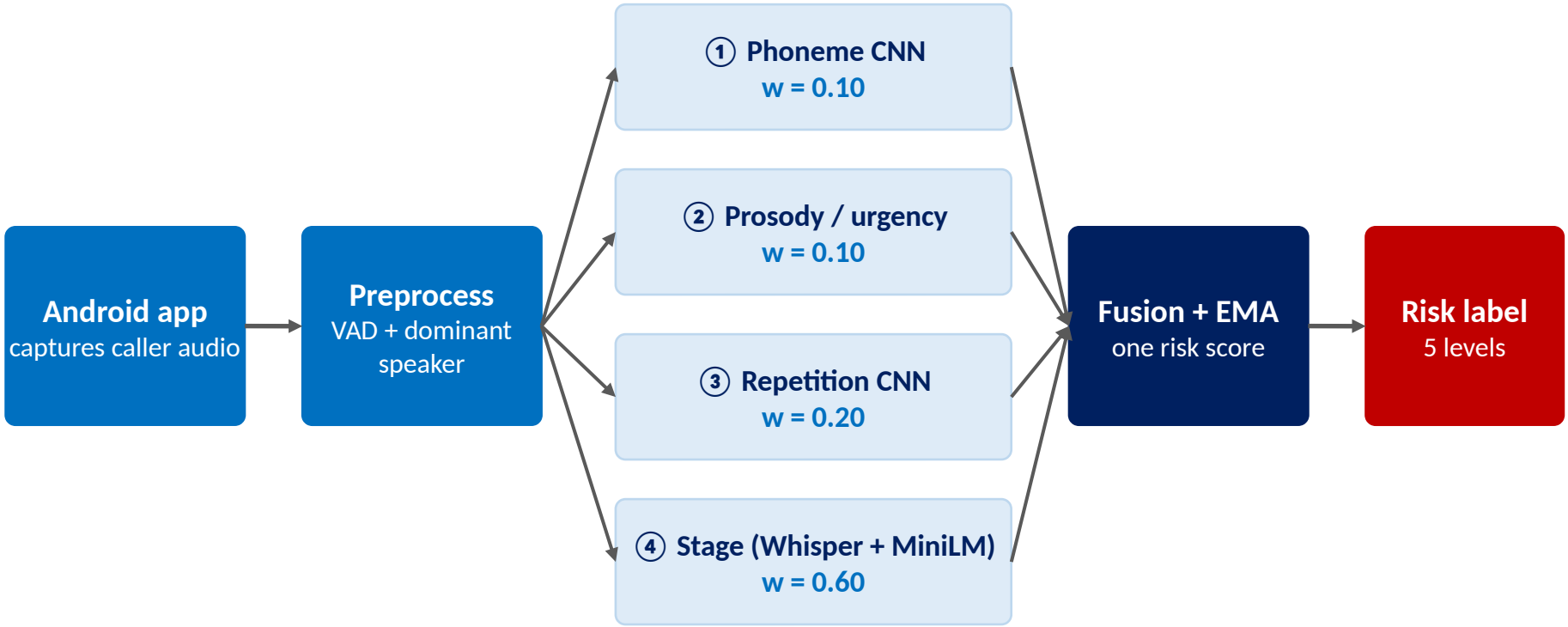
4 **Working open system**

Android app + FastAPI server + public code

Proposed Framework

From raw audio to a live risk score

System Architecture



Every 4 s: 16 kHz · 16-bit PCM over WebSocket → 4 models in parallel → label back to the phone

Step 1: Cleaning the Audio

Normalise
int16 → [−1, 1]

$$x[n] = \frac{s[n]}{32768}, \qquad n = 0, \ \ldots, 63999$$

Voice activity [25]
30 ms frames; skip window if speech < 1 s

$$\Lambda_\ell = \log \frac{p(\mathbf{x}_\ell \mid \text{speech})}{p(\mathbf{x}_\ell \mid \text{noise})} > \tau$$

Frame energy (RMS)
loudness of frame t

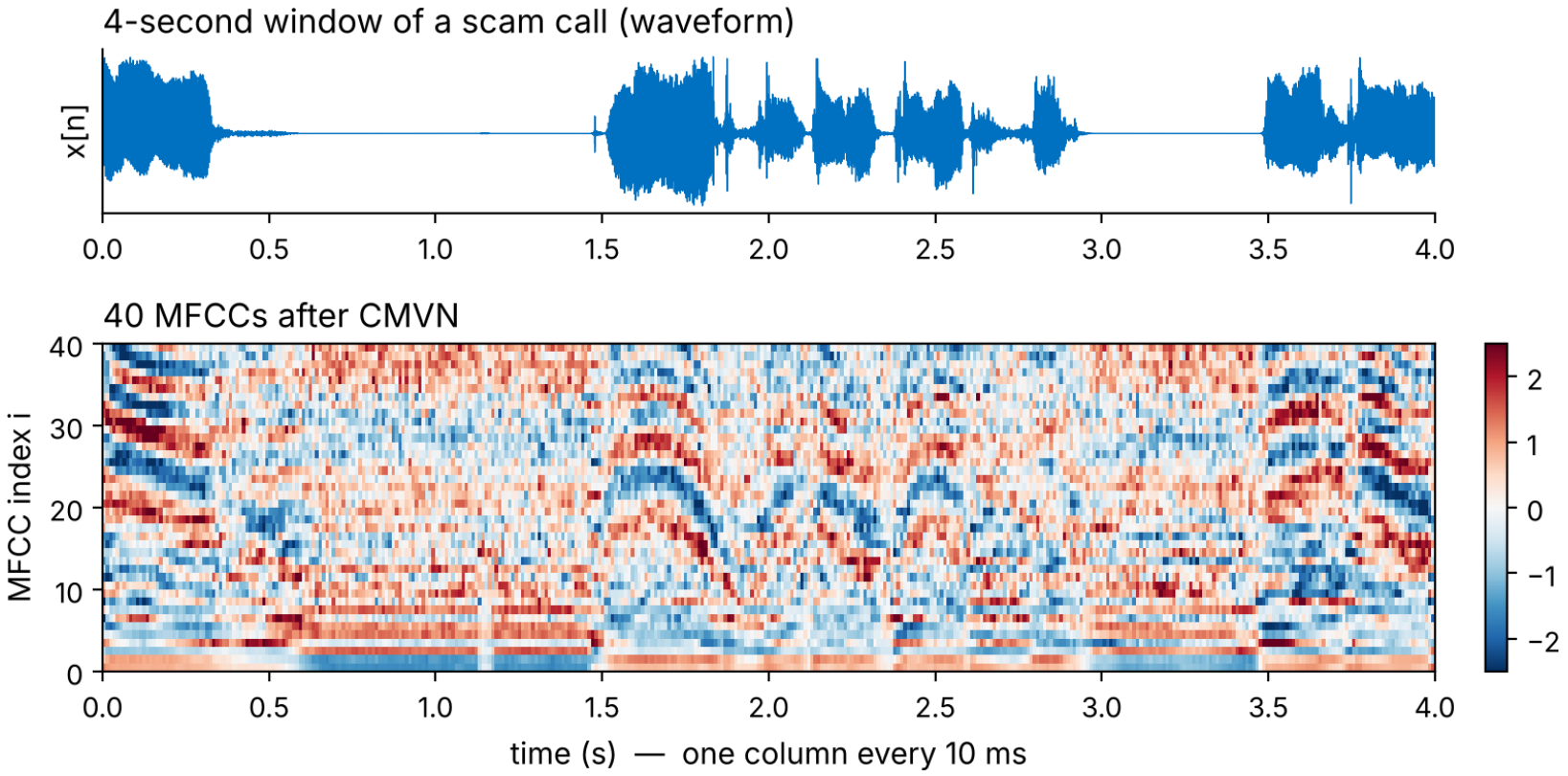
$$E_t = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x_t[n]^2}$$

Dominant speaker
runs ≥ 1.2 s; else fall back to VAD

$$\text{keep } t \quad \text{if} \quad E_t > 0.6\,\bar{E}$$

Step 2: From Sound to MFCC Features

Frames 25 ms → FFT → Mel filters → log → DCT → + Δ, ΔΔ



Phoneme CNN input per window: 370 × 120 (40 MFCC + 40 Δ + 40 ΔΔ) [7], [9]

MFCC: The Maths

Framing
N = 400, H = 160

$$x_t[n] = x[tH + n]\, w[n]$$

Power spectrum
FFT of each frame

$$P_t(k) = \left| \sum_{n=0}^{N-1} x_t[n]\, e^{-j2\pi kn/N} \right|^2$$

Mel scale [8]
ear-like frequency axis

$$m(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$

Filterbank → log → DCT
keep i = 0 … 39

$$c_t(i) = \sum_{m=1}^{M} \log S_t(m) \cos\left[\frac{\pi i\,(m - \frac{1}{2})}{M}\right]$$

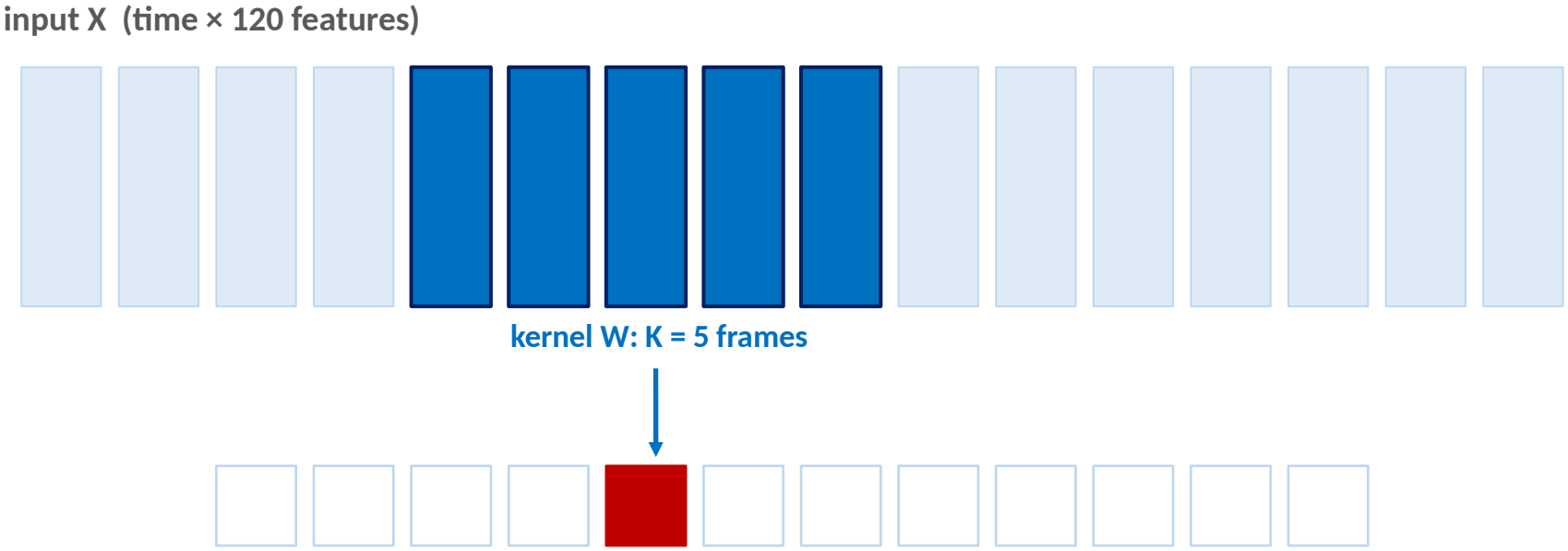
Deltas [9]
change over time

$$\Delta c_t = \frac{(c_{t+1} - c_{t-1}) + 2\,(c_{t+2} - c_{t-2})}{10}$$

Normalise (CMVN)
per coefficient

$$\hat{c}_t = \frac{c_t - \mu}{\sigma}$$

CNN Basics: 1-D Convolution



$$h_{t,\,f} = \mathrm{ReLU}\left(b_f + \sum_{k=0}^{K-1} \sum_{c=1}^{C} W_{f,\,k,\,c}\; x_{t+k,\;c} \right)$$

- **Weight sharing:** same W at every time step [10]
- **Output length:** T′ = T − K + 1 (no padding)

CNN Building Blocks: The Maths

ReLU [11]	$\mathrm{ReLU}(z) = \max(0, z)$	non-linearity
BatchNorm [12]	$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$	stable training
MaxPool	$p_t = \max(h_{2t},\ h_{2t+1})$	halve length
Dropout [13]	$\tilde{h} = \frac{m \odot h}{1 - q}, \quad m_i \sim \mathrm{Bernoulli}(1 - q)$	stop overfitting
Global avg pool [14]	$g_f = \frac{1}{T} \sum_{t=1}^{T} h_{t,f}$	fixed-size vector
Sigmoid	$p = \sigma(z) = \frac{1}{1 + e^{-z}}$	probability (0, 1)

How the CNNs Are Trained

Binary cross-entropy loss (y = 1 scam, 0 normal)

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

Gradient at the output = the error

$$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i$$

Adam optimiser [15]

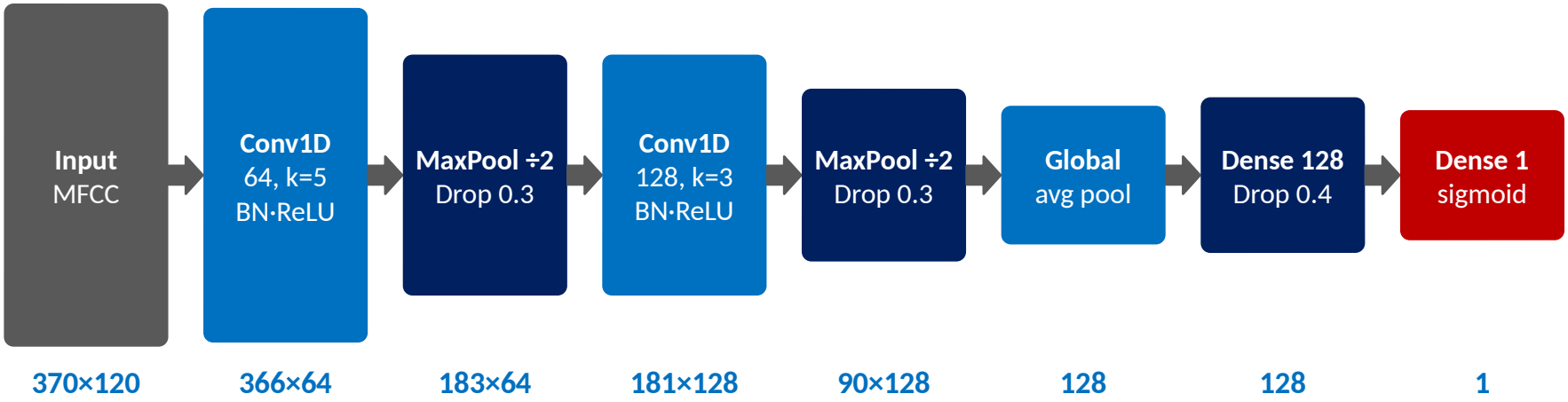
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$\eta = 5\times10^{-4}$ (phoneme CNN) · 10^{-3} (repetition CNN) · $\beta_1 = 0.9$, $\beta_2 = 0.999$

Model 1: Phoneme CNN

$$p_{\text{phon}} = f_\theta(\mathbf{X}) \in (0,1), \qquad \mathbf{X} \in \mathbb{R}^{370 \times 120}$$



≈ 80.6 k

parameters → fast

≈ 120 ms

receptive field ≈ one phoneme

Why CNN?

local patterns, shared weights [10]

Model 2: Prosody Features

Pitch mean & variation
raised, agitated voice

$$\mu_{f_0} = \frac{1}{|V|}\sum_{t \in V} f_0(t), \quad \sigma_{f_0} = \sqrt{\frac{1}{|V|}\sum_{t \in V}(f_0(t) - \mu_{f_0})^2}$$

Loudness (RMS)
speaking forcefully

$$\bar{E} = \frac{1}{T}\sum_{t=1}^{T} E_t$$

Spectral centroid
sharp, tense tone

$$SC_t = \frac{\sum_k f_k \, |X_t(k)|}{\sum_k |X_t(k)|}$$

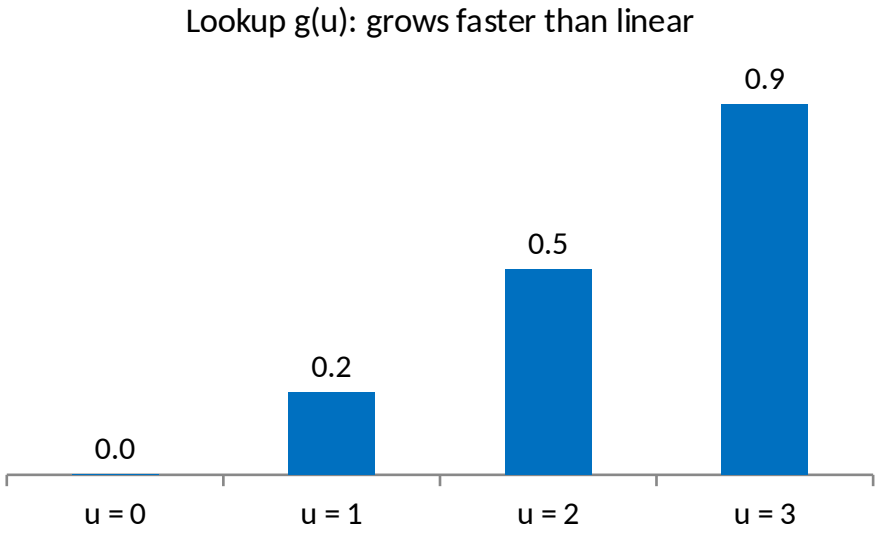
Speech rate
rushing the victim

$$\mathrm{SR} = \frac{\#\,\text{onset peaks}}{\text{duration (s)}}$$

Model 2: Urgency Score

$$u = \min\left(3,\ \sum_{i=1}^{5} \mathbf{1}[f_i > \lambda_i\, b_i]\right)$$

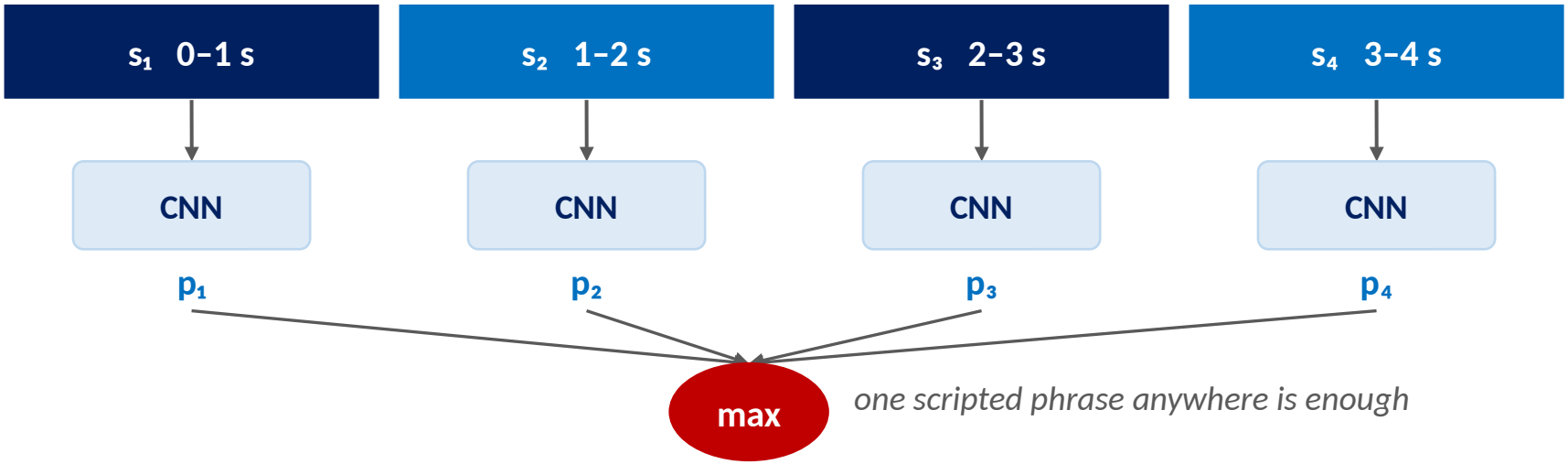
Cue f_i	Factor λ_i
RMS energy	1.8
Pitch mean	1.3
Pitch variation	1.5
Speech rate	1.5
Spectral centroid	1.25



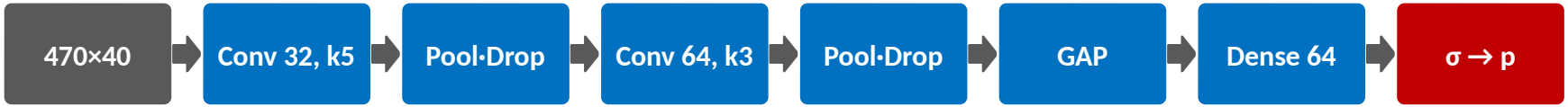
$$r_{\text{urg}} = g(u), \qquad g:\ 0 \mapsto 0,\ 1 \mapsto 0.2,\ 2 \mapsto 0.5,\ 3 \mapsto 0.9$$

b_i = baseline from normal speech · silence gate: $\bar{E} < 0.005 \Rightarrow u = 0$

Model 3: Script-Repetition CNN

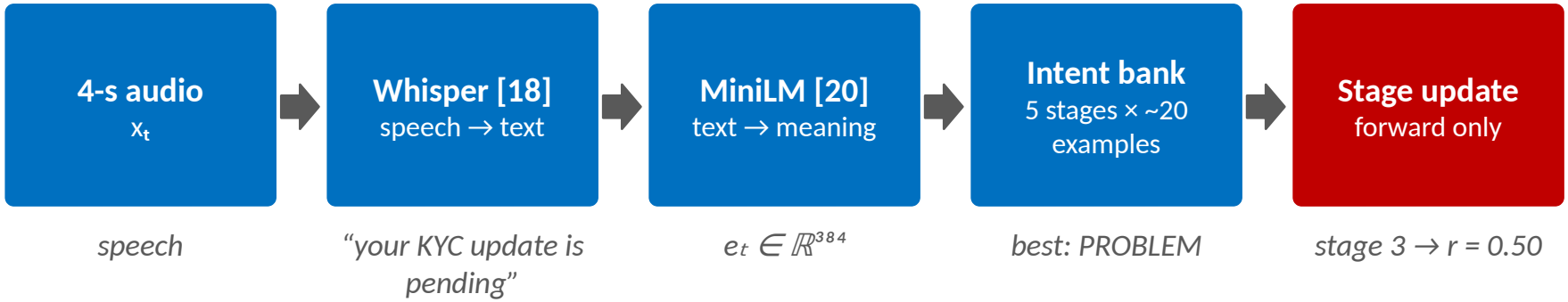


$$p_{\text{rep}} = \max_{j=1,\ldots,4} g_\phi(\text{MFCC}_{40}(s_j))$$



≈ 17.2 k parameters · augmentation: time-stretch 0.85× / 1.15×, pitch ±2 semitones, volume 0.8–1.2× · keyword-spotting CNN idea [16]

Model 4: Conversation Stage



illustrative example

$$\mathbf{e}_t = \mathrm{MiniLM}(\mathrm{Whisper}(x_t)) \in \mathbb{R}^{384}$$

- **Meaning, not exact words:** a rephrased script still matches
- **Intent bank:** English and Hindi examples, for Indian users

Inside Whisper and MiniLM

Whisper [18]: speech → text

- 80-channel log-Mel input
- **CNN stem:** 2 × Conv1D (k = 3, GELU)
- Encoder–decoder Transformer → text

MiniLM-L6-v2 [19], [20]: text → meaning

- 6 Transformer layers, 384-d
- Mean pooling over tokens:

$$\mathbf{e} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{h}_i$$

Self-attention [17]: every word looks at every other word

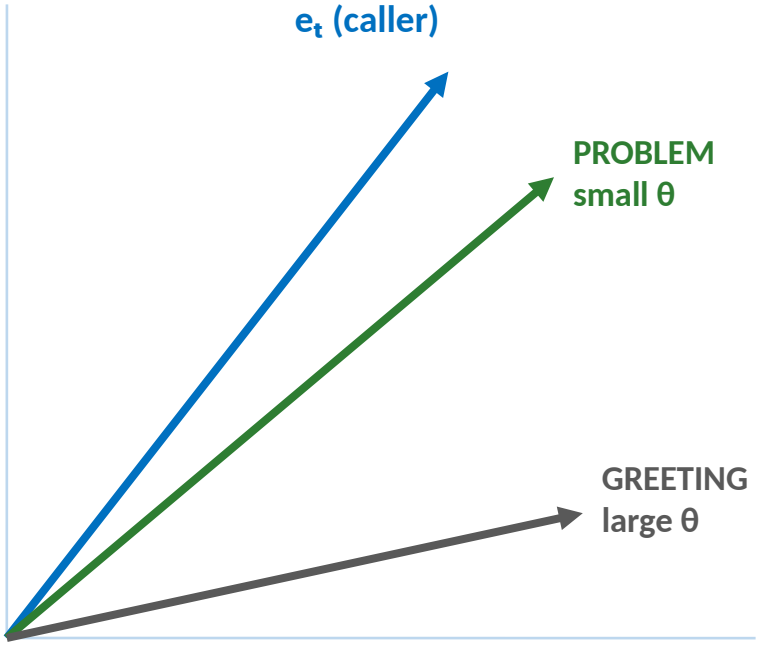
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Matching Meaning: Cosine Similarity

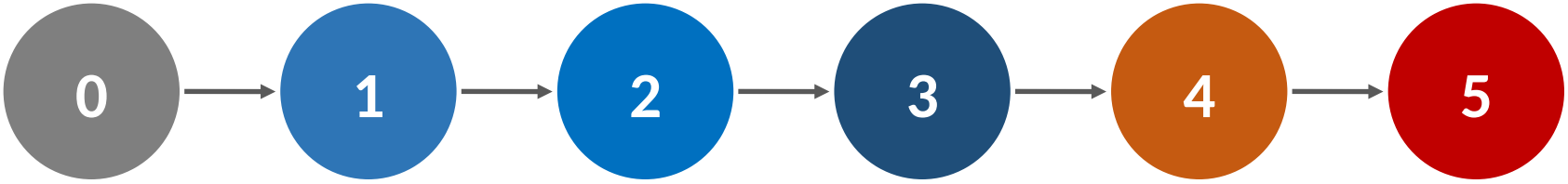
$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \cos\theta$$

$$s_i = \max_j \; \cos(\mathbf{e}_t, \mathbf{e}_{i,j}), \qquad i^* = \arg\max_i \; s_i$$



- Score each stage by its **best** example
- Pick the highest-scoring stage i*
- Accept only if s(i*) ≥ **0.55**
- Small talk below the gate is ignored

Stage Tracking: A Forward-Only State Machine



$$\text{stage}_t = \max\left(\text{stage}_{t-1},\ \pi(i^*)\right) \quad \text{if} \quad s_{i^*} \geq 0.55$$

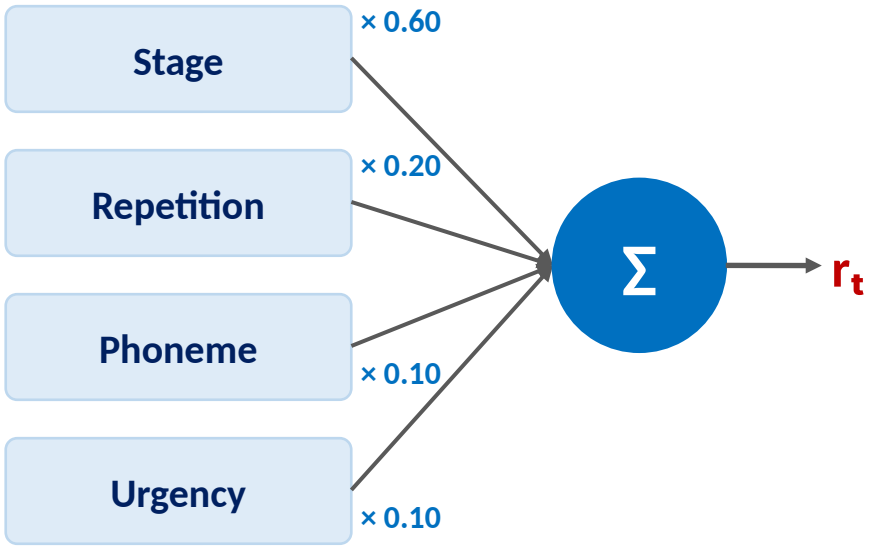
otherwise the stage stays the same

$$r_{\text{stage}} = R(\text{stage}_t)$$

- Scam scripts move forward, so the stage never steps back
- One innocent sentence cannot hide earlier pressure

Ensemble: Weighted Late Fusion

$$r_t = 0.60\, r_{\text{stage}} + 0.20\, p_{\text{rep}} + 0.10\, p_{\text{phon}} + 0.10\, r_{\text{urg}}$$



Input	value	× w	=
Stage (PROBLEM)	0.50	0.60	0.30
Repetition	0.70	0.20	0.14
Phoneme	0.60	0.10	0.06
Urgency (u = 2)	0.50	0.10	0.05
Raw risk r_t			0.55

illustrative values

$$r_t = w_1 p_1 + \ \ldots\ + w_4 p_4, \qquad w_1 + \ \ldots\ + w_4 = 1 \ \Rightarrow\ r_t \in [0, 1]$$

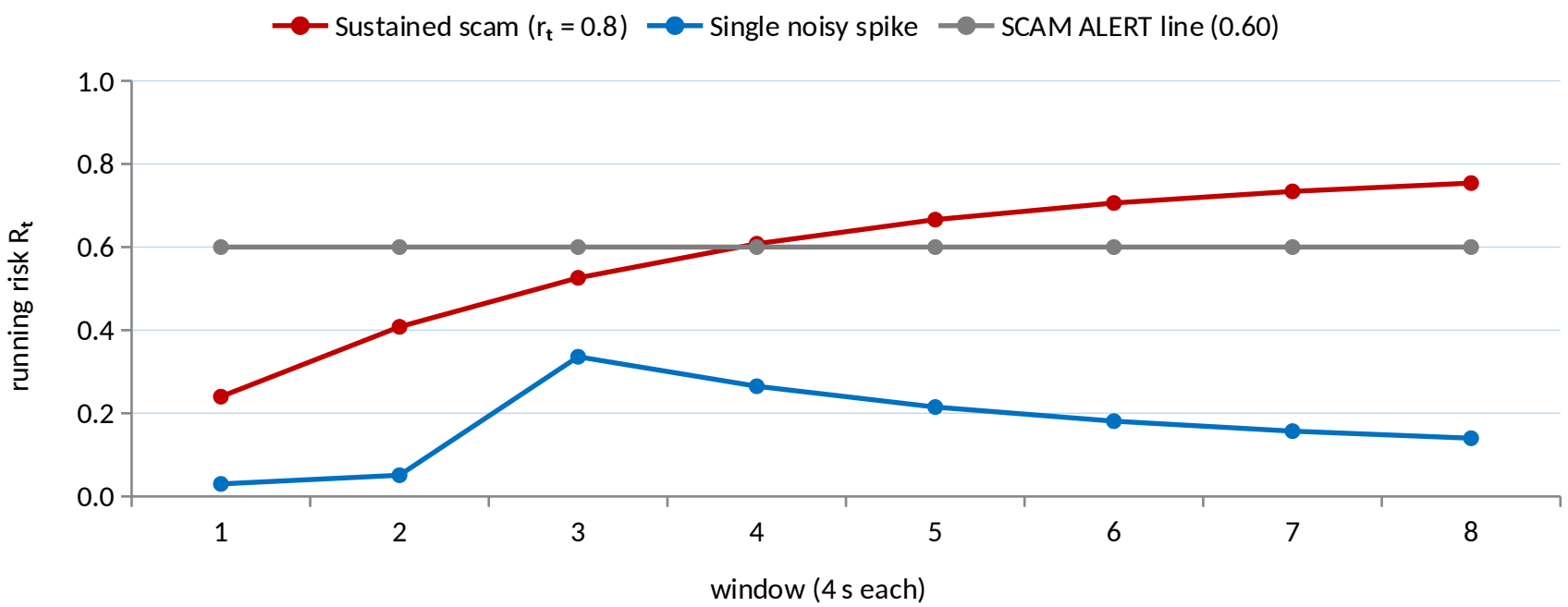
Weighted soft voting [21]: the stage gets the largest weight because it is the most reliable cue

Smoothing Over Time: EMA

$$R_t = \alpha R_{t-1} + \beta r_t, \qquad \alpha = 0.7,\ \beta = 0.3$$

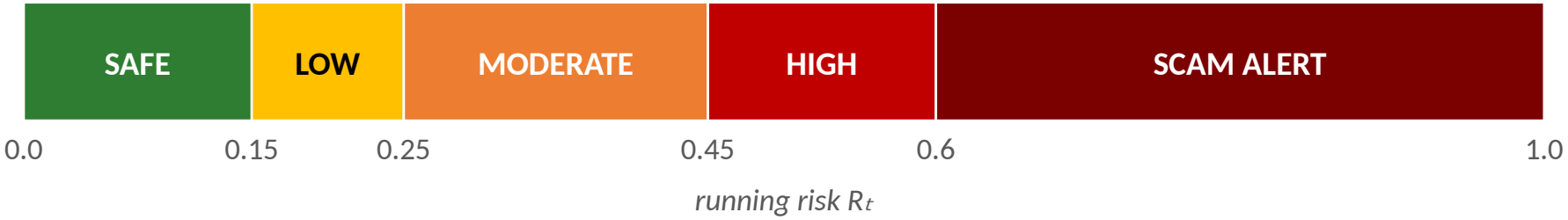
$$R_t = \beta \sum_{k=0}^{t-1} \alpha^k r_{t-k} \qquad (R_0 = 0)$$

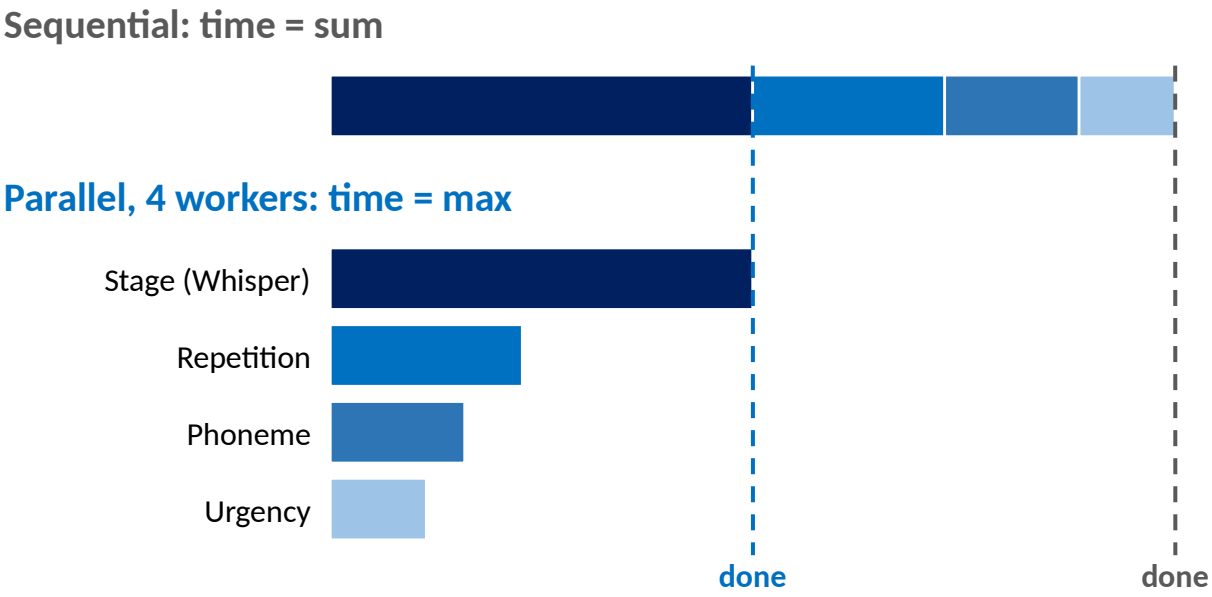
$$\text{memory} \approx \frac{1}{1-\alpha} \approx 3.3 \text{ windows} \approx 13\,\text{s}$$



Spike → only 0.34 (MODERATE), then fades · Sustained scam crosses 0.60 at window 4 (16 s) · [22]

Risk Levels and Real-Time Execution





$$T_{\text{window}} = \max(T_1, T_2, T_3, T_4) \; < \; T_1 + T_2 + T_3 + T_4$$

Results and Discussion

Does it work, and where are its limits?

Dataset and Evaluation Setup

Training data (built by us, public)

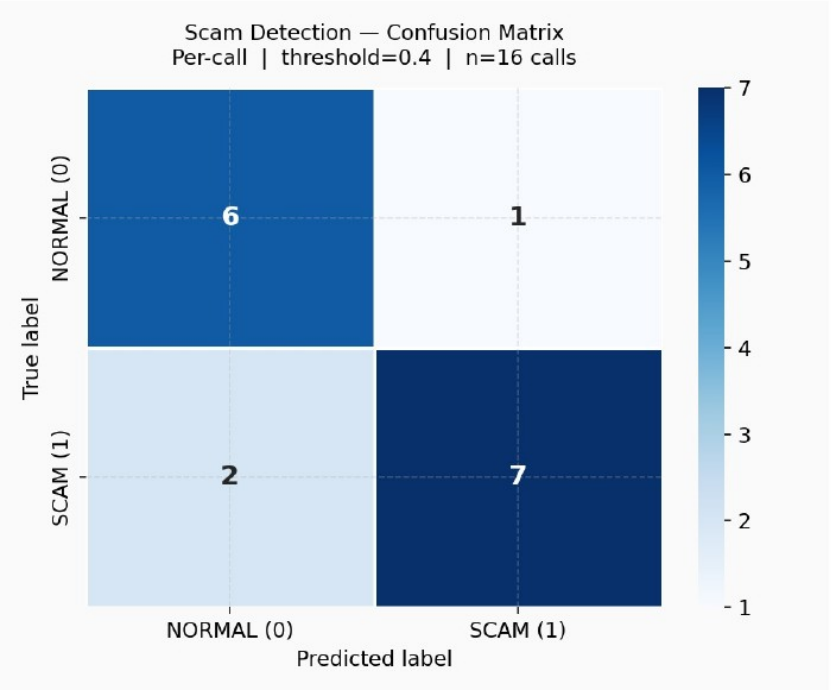
- No public real-time scam-call dataset exists
- Phoneme & urgency: ≈ 28 min scam + ≈ 23 min normal
- Repetition: ≈ 80 clips × 15 keywords + 800–900 negatives
- Stage model: intent example sentences

16 full calls: 7 normal, 9 scam

Same pipeline as live deployment

Final R_t of each call = its score

From Confusion Matrix to Metrics



$$\text{Accuracy} = \frac{TP + TN}{N} = \frac{7 + 6}{16} = 0.8125$$

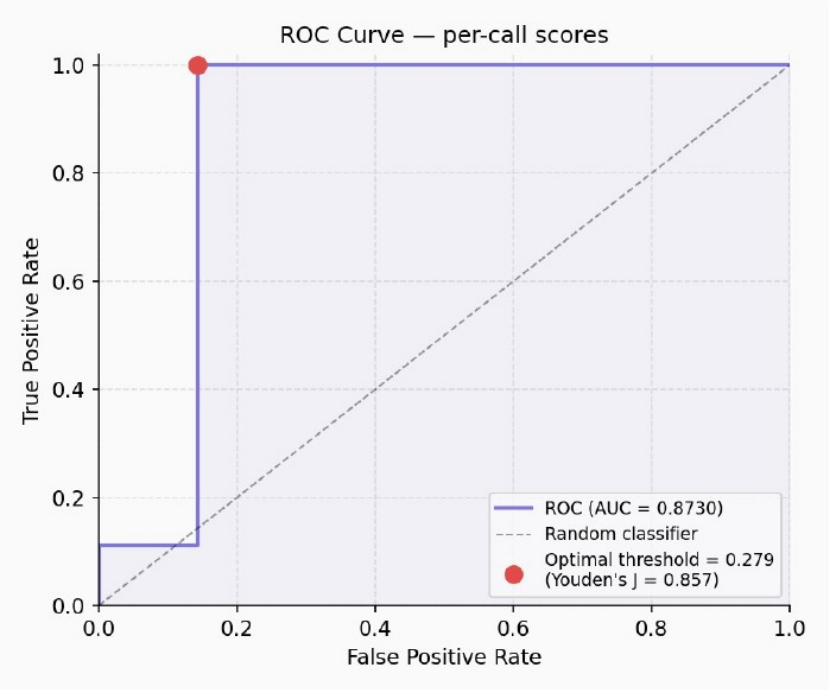
$$\text{Precision} = \frac{TP}{TP + FP} = \frac{7}{8} = 0.875$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{7}{9} = 0.778$$

$$F_1 = \frac{2PR}{P + R} = 0.824$$

- TP = 7 caught · FN = 2 missed · FP = 1 false alarm · TN = 6
- **High precision** → few false alarms → users keep trusting the alert

ROC Curve, AUC and the Threshold



$$\mathrm{TPR} = \frac{TP}{TP + FN}, \qquad \mathrm{FPR} = \frac{FP}{FP + TN}$$

$$J = \mathrm{TPR} - \mathrm{FPR}$$

Threshold	TPR	FPR	J
0.279 (max J)	1.000	0.143	0.857
0.40 (reported)	0.778	0.143	0.635

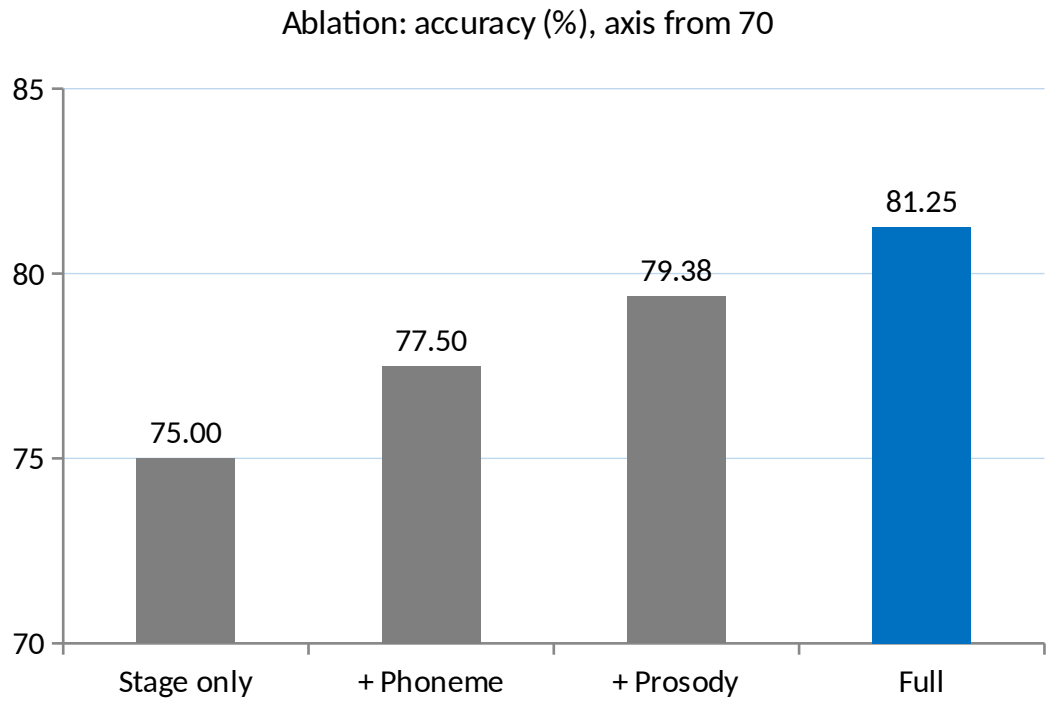
0.279 is tuned on the same 16 calls, so we report the stricter 0.40.

AUC = chance a random scam call scores higher than a random normal call [23], [24]

$$\mathrm{AUC} = P(s_{\mathrm{scam}} > s_{\mathrm{normal}}) = \frac{1}{n_+ n_-} \sum_{i \in +} \sum_{j \in -} \mathbf{1}[s_i > s_j]$$

Results and Ablation Study

81.25%	0.813	0.873	0.875
Accuracy	Weighted F1	ROC-AUC	Scam precision



Take-aways

- Stage alone: already 75%
- Every module adds accuracy
- Full ensemble: +6.25 points

Strengths and Limitations

Strengths

- Understands **intent**, not keywords
- Live risk **every 4 s**
- Four modalities back each other up
- EMA → few false alarms (87.5% precision)
- Complete, **open** system

Limitations → our answer

- 16 test calls → proof of concept; no public dataset exists
- Whisper errors on noise → 3 acoustic models + EMA
- Hand-set urgency thresholds → explainable; learn next
- Needs internet → on-device models next

Conclusion and Future Work

- Scam calls follow a script; its **stage** reveals intent
- Four models + **weighted fusion** + **EMA** → stable live risk
- **81.25%** accuracy · **0.873** ROC-AUC · every module helps

Future work

- Larger multilingual dataset · on-device inference
- Learned fusion weights · telecom integration

Thank you · Questions?

References (1/2)

[1] R. S. S. Kumar et al., “Scam calls detection using machine learning approaches,” in Proc. IEEE Int. Conf. Advanced Machine Learning and Artificial Intelligence, 2023, pp. 1–6.

[2] M. Young and A. West, “Evaluating the limitations of rule-based fraud detection systems,” J. Telecommunication Security, vol. 14, no. 3, pp. 112–120, 2019.

[3] N. Hidayati, M. I. Herdiansyah, and A. S. R. M. Sinaga, “Fraud detection call detail record using machine learning in telecommunications company,” Adv. Sci. Technol. Eng. Syst. J., vol. 5, no. 4, pp. 9–16, 2020.

[4] X. Yin, Y. Li, and Z. Chen, “Temporal pattern recognition in fraudulent call detection using dynamic time warping,” IEEE Access, vol. 8, pp. 45012–45022, 2020.

[5] B. Hong, T. Connie, T. S. Ong, and A. B. J. Teoh, “Classifying scam calls through content analysis with dynamic sparsity top-k attention regularization,” IEEE Access, vol. 13, pp. 111733–111751, 2025.

[6] I. Wood, M. Kepkowski, L. Zinatullin, T. Darnley, and M. A. Kaafar, “An analysis of scam baiting calls: Identifying and extracting scam stages and scripts,” arXiv:2307.01965, 2023.

[7] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” IEEE Trans. ASSP, vol. 28, no. 4, pp. 357–366, 1980.

[8] S. S. Stevens, J. Volkmann, and E. B. Newman, “A scale for the measurement of the psychological magnitude pitch,” J. Acoust. Soc. Am., vol. 8, no. 3, pp. 185–190, 1937.

[9] S. Furui, “Speaker-independent isolated word recognition using dynamic features of speech spectrum,” IEEE Trans. ASSP, vol. 34, no. 1, pp. 52–59, 1986.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[11] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in Proc. ICML, 2010, pp. 807–814.

[12] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. ICML, 2015, pp. 448–456.

References (2/2)

[13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” JMLR, vol. 15, pp. 1929–1958, 2014.

[14] M. Lin, Q. Chen, and S. Yan, “Network in network,” in Proc. ICLR, 2014.

[15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.

[16] T. N. Sainath and C. Parada, “Convolutional neural networks for small-footprint keyword spotting,” in Proc. Interspeech, 2015, pp. 1478–1482.

[17] A. Vaswani et al., “Attention is all you need,” in Proc. NeurIPS, 2017, pp. 5998–6008.

[18] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in Proc. ICML, 2023, pp. 28492–28518.

[19] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[20] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in Proc. NeurIPS, 2020.

[21] J. Kittler, M. Hatef, R. P. W. Duin, and J. Matas, “On combining classifiers,” IEEE Trans. PAMI, vol. 20, no. 3, pp. 226–239, 1998.

[22] S. W. Roberts, “Control chart tests based on geometric moving averages,” Technometrics, vol. 1, no. 3, pp. 239–250, 1959.

[23] T. Fawcett, “An introduction to ROC analysis,” Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[24] W. J. Youden, “Index for rating diagnostic tests,” Cancer, vol. 3, no. 1, pp. 32–35, 1950.

[25] Google, “WebRTC voice activity detector (VAD),” WebRTC open-source project; used via the py-webrtcvad interface.